Object-Oriented Programming Cont..

1. Quick recap: classes and objects

A class is a blueprint; an object is an instance created from that blueprint.

For example, `Student` is a class, while `s1` and `s2` are individual student objects.

```
class Student {
public:
    string name;
    int marks;

    void display() {
        cout << name << " " << marks << endl;
    }
};

int main() {
    Student s1;
    s1.name = "Riya";
    s1.marks = 90;

    s1.display();
}
```

2. The four pillars of OOP

The four pillars are:

1. Encapsulation: Bundle data and methods; control access.

Example: bank account balance.

2. Abstraction: Expose essential operations; hide implementation details.

Example: ATM.

3. Inheritance: Create a specialized class from an existing class.

Example: Dog inherits from Animal.

4. Polymorphism : One interface, different behaviours.

Example: different implementations of `speak()`.

2.1 Encapsulation

Definition: Encapsulation combines data and the functions that operate on that data inside a class, while controlling access to the data.

Real-life example: bank account

Imagine a bank account with ₹5,000.

If the balance were publicly accessible, any code could change it to −₹10,000. A better design allows users to deposit and withdraw through controlled functions.

```
class BankAccount {
private:
    double balance = 0;

public:
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    bool withdraw(double amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }

        balance -= amount;
        return true;
    }

    double getBalance() {
        return balance;
    }
};
```

Explanation:

- `private`: outside code cannot directly access `balance`.

- `public`: outside code can call `deposit()`, `withdraw()` and `getBalance()`.
- Validation prevents invalid transactions.
- The class keeps its data and related behaviour together.

Important distinction

Encapsulation is not just making every variable private. It is about bundling state and behaviour and controlling how that state is accessed or modified.

2.2 Abstraction

Definition: Abstraction means showing only the essential features to the user while hiding the unnecessary implementation details.

Real-life example: ATM

When you withdraw money, you:

1. Insert your card.
2. Enter your PIN.
3. Select an amount.
4. Receive cash.

You do not need to know how the ATM communicates with the bank server or verifies the transaction internally.

```
class ATM {
public:
    void withdrawMoney() {
        verifyUser();
        checkBalance();
        dispenseCash();
    }

private:
    void verifyUser() {
        cout << "Verifying user\n";
    }

    void checkBalance() {
        cout << "Checking balance\n";
    }

    void dispenseCash() {
```

```
        cout << "Dispensing cash\n";
    }
};
```

The user calls `withdrawMoney()` without needing to understand each internal step.

How is abstraction implemented in C++?

- Classes and access specifiers.
- Public interfaces that hide internal implementation.
- Abstract classes and pure virtual functions.

Encapsulation vs abstraction

Encapsulation	Abstraction
Bundles data and behaviour	Hides implementation complexity
Controls access to internal state	Exposes essential operations
Example: private bank balance	Example: ATM withdrawal interface

Easy memory trick: Encapsulation = protect and bundle. Abstraction = simplify and hide.

2.3 Inheritance

Definition: Inheritance allows a new class, called the derived class, to reuse and extend accessible members of an existing base class.

Real-life example

A Dog is an Animal. A dog can perform general animal behaviours, such as eating, and can also have its own behaviours, such as barking.

```
class Animal {
public:
    void eat() {
```

```
        cout << "Eating\n";
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Woof!\n";
    }
};

int main() {
    Dog d;
    d.eat();   // Inherited from Animal
    d.bark();  // Defined in Dog
}
```

Types of inheritance

1. Single inheritance

One base class and one derived class.

A → B

2. Multilevel inheritance

A class inherits from a derived class.

A → B → C

3. Multiple inheritance

One class inherits from multiple base classes.

A + B → C

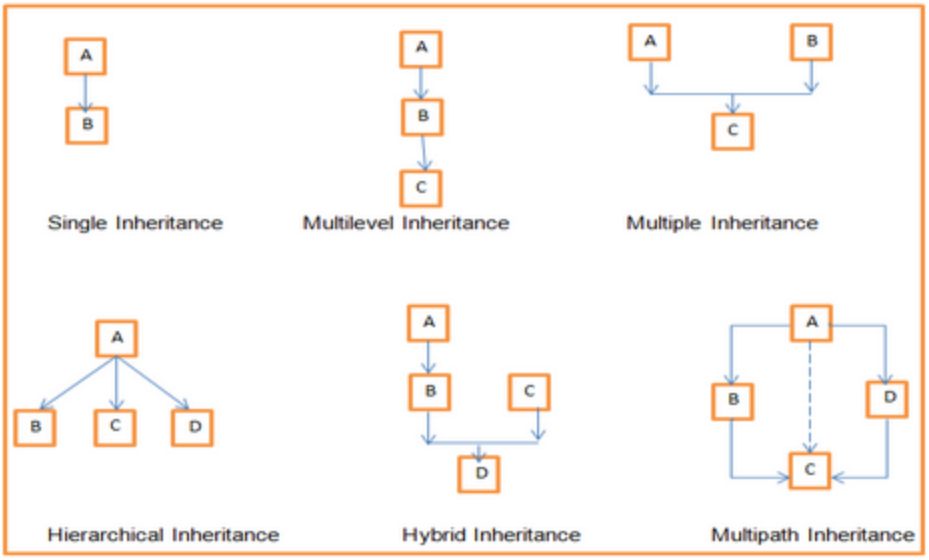
4. Hierarchical inheritance

Multiple classes inherit from one base class.

A → B and A → C

5. Hybrid inheritance

A combination of two or more inheritance types.



2.4 Polymorphism

Definition: Polymorphism means “many forms.” The same interface can represent different behaviours.

There are two major forms.

A. Compile-time polymorphism

The compiler selects the relevant operation based on the code and arguments.

Common examples:

- Function overloading.
- Operator overloading.

Function overloading

```
int add(int a, int b) {
   return a + b;
}

double add(double a, double b) {
   return a + b;
}

int add(int a, int b, int c) {
   return a + b + c;
}
```

All three functions are called `add`, but their parameter lists differ.

Important: You cannot overload functions by changing only their return types.

B. Run-time polymorphism

The implementation is selected at run time through virtual functions and overriding.

```
class Animal {
public:
    virtual void speak() {
        cout << "Animal sound\n";
    }

    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void speak() override {
        cout << "Woof!\n";
    }
};

int main() {
    Dog d;
    Animal* p = &d;

    p->speak();  // Woof!
}
```

Explanation:

1. `d` is a Dog object.
2. `p` is a pointer of type `Animal*`.
3. The pointer refers to the Dog object.
4. Since `speak()` is virtual, the Dog implementation is called.

If `speak()` were not virtual, the call through `Animal*` would use the base-class implementation.

The `override` keyword helps the compiler detect mistakes in the derived method's signature.

Overloading vs overriding

Overloading	Overriding
Same function name, different parameter lists	Derived class redefines a virtual base-class function
Usually compile-time polymorphism	Run-time polymorphism through a base pointer/reference
Can occur within one class	Requires inheritance
Return type alone cannot distinguish overloads	Matching virtual function signature is important

Recursion

What is recursion?

Recursion is when a function calls itself to solve a smaller version of the same problem.

Imagine opening a box containing a smaller box, which contains another smaller box. You continue until you reach the final box, then work backwards.

Every recursive solution needs:

1. **Base case: when should the function stop?**
2. **Recursive case: how does it call itself?**
3. **Progress: how does each call move closer to the base case?**
4. **Return/unwinding: how do the waiting calls finish?**

Visualising a recursive call

```
fact(4)
```

```
4 × fact(3)
```

`4 × 3 × fact(2)`

`4 × 3 × 2 × fact(1)`

Base case: `fact(1) = 1`

The calls now return in reverse order, producing 1 → 2 → 6 → 24.

A function call stores information such as its local variables and return location on the call stack. If recursion never reaches a base case, the program may eventually run out of stack space.

Q1 Factorial

Factorial is defined as:

$$n! = n \times (n-1) \times \cdots \times 1$$

And $0! = 1$.

```
long long fact(int n) {
    if (n <= 1) {
        return 1;
    }

    return 1LL * n * fact(n - 1);
}
```

Dry run: `fact(4)`

Call	Operation
`fact(4)`	`4 * fact(3)`
`fact(3)`	`3 * fact(2)`

`fact(2)` `2 * fact(1)`

`fact(1)` Returns `1`

Now the answers return:

- `fact(2) = 2 × 1 = 2`
- `fact(3) = 3 × 2 = 6`
- `fact(4) = 4 × 6 = 24`

Complexity: Time $O(n)$, call-stack space $O(n)$.

Common mistake: calling `fact(n)` again with the same `n`. The recursive argument must get smaller.

Q2 — Fibonacci

The standard Fibonacci sequence is:

$$F(0) = 0, F(1) = 1$$

$$F(n) = F(n-1) + F(n-2)$$

```
long long fib(int n) {
   if (n == 0) return 0;
   if (n == 1) return 1;

   return fib(n - 1) + fib(n - 2);
}
```

Dry run: `fib(4)`

$$fib(4) = fib(3) + fib(2)$$

$$= (fib(2) + fib(1)) + (fib(1) + fib(0))$$

$$= (1 + 1) + (1 + 0) = 3$$

The first values are 0, 1, 1, 2, 3, 5, 8, 13…

Q3 Sum of an array

Define `sumArray(a, n)` to mean the sum of the first `n` elements.

The key idea is:

- If `n == 0`, the sum is zero.
- Otherwise, add the last included element to the sum of the first `n-1` elements.

```
int sumArray(const vector<int>& a, int n) {
   if (n == 0) {
      return 0;
   }

   return a[n - 1] + sumArray(a, n - 1);
}
```

Example:

```
vector<int> a = {2, 4, 6};

cout << sumArray(a, (int)a.size());  // 12
```

Dry run

`sumArray([2,4,6], 3)`

$$= 6 + sumArray(a, 2)$$

$$= 6 + 4 + sumArray(a, 1)$$

$$= 6 + 4 + 2 + sumArray(a, 0)$$

$$= 6 + 4 + 2 + 0 = 12$$

Complexity: Time $O(n)$, stack space $O(n)$.

The `const vector<int>&` parameter avoids copying the entire vector in every recursive call.

Generate all subsequences

This is a key recursion topic because it introduces decision trees and backtracking.

A subsequence preserves the original order of selected elements, but the selected elements do not need to be adjacent.

For example, for `[1,2,3]`, `[1,3]` is a subsequence, but `[3,1]` is not.

For `[1,2]`, the subsequences are:

- — empty subsequence
- `[1]`
- `[2]`
- `[1,2]`

For an array of size n, there are 2^n subsequences, including the empty one.

Core idea: include or exclude

For each element, make two decisions:

1. Include the current element.
2. Exclude the current element.

```
void generate(int i,
              const vector<int>& a,
              vector<int>& path) {

    if (i == (int)a.size()) {
        cout << "[ ";
        for (int x : path) {
            cout << x << " ";
        }
        cout << "]\n";
        return;
    }

    // Choice 1: include a[i]
    path.push_back(a[i]);
    generate(i + 1, a, path);

    // Undo the choice
    path.pop_back();
```

```
    // Choice 2: exclude a[i]
    generate(i + 1, a, path);
}
int main() {
    vector<int> a = {1, 2, 3};
    vector<int> path;

    generate(0, a, path);
}
```

- `push_back(a[i])` includes the current element.
- The first recursive call explores all possibilities that include it.
- `pop_back()` removes that choice.
- The second recursive call explores possibilities that exclude it.

This process is called backtracking: make a choice, explore it, undo the choice, and explore the alternative.

Why is `pop_back()` necessary? Because the same vector is reused across branches. Without removing the previous choice, the next branch would inherit an incorrect state.

Complexity:

- Time: $O(n \cdot 2^n)$ when printing every subsequence.
- Auxiliary space: $O(n)$ for the recursion stack and current path.

Common student mistakes

- Printing at every recursive call instead of only at the base case.
- Forgetting `pop_back()`.
- Forgetting to increase `i`.
- Confusing a subsequence with a contiguous subarray.

9. Coin toss using recursion

For every coin toss, there are two possible outcomes: `H` or `T`.

For n tosses, the total number of possible sequences is:

$$2^n$$

For two tosses: `HH`, `HT`, `TH`, `TT`.

For three tosses: `HHH`, `HHT`, `HTH`, `HTT`, `THH`, `THT`, `TTH`, `TTT`.

C++ implementation

```
void coinToss(int n, string& path) {
    if ((int)path.size() == n) {
        cout << path << "\n";
        return;
    }

    // Choice 1: Heads
    path.push_back('H');
    coinToss(n, path);
    path.pop_back();

    // Choice 2: Tails
    path.push_back('T');
    coinToss(n, path);
    path.pop_back();
}
```

Call it:

```
int main() {
    string path;
    coinToss(3, path);
}
```

Explanation:

For every position, the function first explores the `H` branch, then the `T` branch. Once the string length reaches `n`, one complete outcome is printed.

Complexity:

- Time: $O(n \cdot 2^n)$ when printing every sequence.
- Recursion depth: $O(n)$.

For fair, independent coin tosses, every length-n sequence has probability $1/2^n$. The code above enumerates all possibilities; it does not simulate random tosses.